



National Textile University

Department of Computer Science

Subject: Operating System

Submitted to: Sir Nasir

Submitted by: Kashmir Jamshaid

Reg. number: 23-NTU-CS-1167

After Mid's first Homework

Semester: 5th

Operating Systems – COC 3071

SE 5th A – Fall 2025

After-mid Homework -1

Part 1:

Semaphore theory

1. A counting semaphore is initialized to 7. If 10 wait() and 4 signal() operations are performed, find the final value of the semaphore.

Solution:

Let the semaphore be represented as keys.

The semaphore starts at 7, which means 7 keys are available.

- Each wait() operation takes 1 key, so 10 wait() operations mean -10 keys.
- Each signal() operation returns 1 key, so 4 signal() operations mean +4 keys.

So,

$$\begin{aligned}\text{Final value} &= \text{Initial keys} - \text{keys taken} + \text{keys returned} \\ &= 7 - 10 + 4 = 1\end{aligned}$$

Final semaphore value is 1.

2. A semaphore starts with value 3. If 5 wait() and 6 signal() operations occur, calculate the resulting semaphore value.

Solution:

Let the semaphore be represented as keys.

- The semaphore starts with **3 keys**.
- Each **wait()** operation takes **1 key**, so **5 wait() operations mean -5 keys**.
- Each **signal()** operation returns **1 key**, so **6 signal() operations mean +6 keys**.

Now combine everything:

$$\text{Final value} = 3 - 5 + 6 = 4$$

The resulting semaphore value is 4.

3. A semaphore is initialized to 0. If 8 signal() followed by 3 wait() operations are executed, find the final value.

Solution:

Let the semaphore be represented as keys.

- The semaphore starts at 0, which means no keys are available.
- Each signal() operation adds 1 key, so 8 signal() operations add +8 keys.
- Each wait() operation takes 1 key, so 3 wait() operations take -3 keys.

Now combine everything:

$$\text{Final value} = 0 + 8 - 3 = 5$$

The final semaphore value is 5.

4. A semaphore is initialized to 2. If 5 wait() operations are executed:

a) How many processes enter the critical section?

b) How many processes are blocked?

Solution:

Let the semaphore be represented as keys.

- The semaphore starts with 2 keys, meaning 2 processes can enter the critical section.
- The first 2 wait() operations get the keys and enter.
- The remaining 3 wait() operations find no keys available, so they must wait (blocked).

a) Number of processes that enter the critical section: 2

b) Number of processes that are blocked: 3

5. A semaphore starts at 1. If 3 wait() and 1 signal() operations are performed:

a) How many processes remain blocked?

b) What is the final semaphore value?

Solution:

The semaphore starts with 1 key, meaning 1 process can enter the critical section.

- The first wait () operation takes the key , Process 1 enters.
- The next 2 wait () operations find no keys available ,Processes 2 and 3 are blocked.
- Then 1 signal () operation returns a key ,Process 2 gets the key and enters.
- So 3rd process remains blocked.

a) Processes that remain blocked: 1 (Process 3)

b) Final semaphore value: -1.

6. semaphore S = 3;

wait(S);

wait(S);

signal(S);

wait(S);

wait(S);

a) How many processes enter the critical section?

b) What is the final value of S?

Solution:

Semaphore starts with 3 keys, meaning 3 processes can enter.

1. First wait(S):

- **Process 1 takes 1 key so,**

$$S = 3 - 1 = 2$$

- **Process 1 enters**

2. Second wait(S):

- **Process 2 takes 1 key so,**

$$S = 2 - 1 = 1$$

- **Process 2 enters**

3. signal(S):

- **A key is returned so ,**

$$S = 1 + 1 = 2 \text{ (increased by 1)}$$

4. Third wait(S):

- **Process 3 takes 1 key so**

$$S = 2 - 1 = 1$$

- **Process 3 enters**

5. Fourth wait(S):

- **Process 4 takes 1 key so**

$$S = 1 - 1 = 0$$

- **Process 4 enters**

a) Processes that enter the critical section: 4

b) Final value of S: 0

7.

semaphore S = 1;

wait(S);

wait(S);

signal(S);

signal(S);

a) How many processes are blocked?

b) What is the final value of S?

Solution:

Semaphore starts with 1 key, meaning 1 process can enter.

1. First wait(S):

➤ Process 1 takes 1 key so ,

$$S = 1 - 1 = 0$$

➤ Process 1 enters

2. Second wait(S):

➤ Process 2 tries to take a key so,
no key available

➤ Process 2 is blocked so,

$$S = 0 - 1 = -1$$

3. First signal(S):

➤ A key is returned

➤ Blocked process 2 gets the key and enters so,

$$S = -1 + 1 = 0$$

4. Second signal(S):

➤ The key is returned so,

$$➤ S = 0 + 1 = 1$$

a) Processes that were blocked: 1 (Process 2)

b) Final value of S: 1

8. A binary semaphore is initialized to 1. Five wait() operations are executed without any signal(). How many processes enter the critical section and how many are blocked?

Solution:

Binary semaphore = 1 key, meaning only 1 process can enter at a time.

1. First wait():

- Process 1 takes the key so , enters the critical section
- Semaphore value = $1-1=0$

2. Next 4 wait():

- No key is available
- Processes 2, 3, 4, 5 are blocked

9. A counting semaphore is initialized to 4. If 6 processes execute wait() simultaneously, how many proceed and how many are blocked?

Solution:

Semaphore = 4 keys, meaning 4 processes can enter the critical section at the same time.

1. First 4 processes take the keys so , enter the critical section

- Semaphore value = 0

2. Remaining 2 processes try to enter so , no keys available

- They are blocked

- **Processes that proceed: 4**
- **Processes that are blocked: 2**

10. A semaphore S is initialized to 2. wait(S); wait(S); wait(S); signal(S); signal(S); wait(S);

a) Track the semaphore value after each operation.

b) How many processes were blocked at any time?

Solution:

wait(S) → Process 1

- Process 1 takes 1 key
- Semaphore value = $2 - 1 = 1$
- Process 1 enters the critical section

wait(S) → Process 2

- Process 2 takes 1 key
- Semaphore value = $1 - 1 = 0$
- Process 2 enters the critical section

wait(S) → Process 3

- No keys left ($S = 0$)
- Process 3 blocked
- Semaphore value = $0 - 1 = -1$

signal(S) → A key returned

- Key is returned
- Process 3 wakes up and takes the key
- Semaphore value = $-1 + 1 = 0$

signal(S) → Another key returned

- No one is waiting now
- Semaphore value = $0 + 1 = 1$

wait(S) → Process 4

- Process 4 takes 1 key

- Semaphore value = $1 - 1 = 0$
- Process 4 enters the critical section

a) Semaphore values after each step:

$2 \rightarrow 1 \rightarrow 0 \rightarrow -1 \rightarrow 0 \rightarrow 1 \rightarrow 0$

b) Processes blocked at any time: 1 (Process 3) .

11. A semaphore is initialized to 0. Three processes execute wait() before any signal(). Later, 5 signal() operations are executed.

a) How many processes wake up?

b) What is the final semaphore value?

Solution:

semaphore starts at 0 keys, meaning no process can enter yet.

3 wait() operations

- **Process 1 does wait() so, no key available and P1 is blocked :**

S = -1

- **Process 2 does wait() so , no key available and P2 is blocked:**

S = -2

- **Process 3 does wait() so ,no key available and P3 is blocked:**

S = -3

All 3 processes are now blocked.

5 signal() operations

Each signal() adds 1 to the semaphore:

- 1. Signal 1 : wakes Process 1 so :**

S = $-3 + 1 = -2$

- 2. Signal 2 : wakes Process 2 so :**

$$S = -2 + 1 = -1$$

3. Signal 3 : wakes Process 3 so :

$$S = -1 + 1 = 0$$

After this, all blocked processes are awake, and $S = 0$, so next two are available now

4. Signal 4 $\rightarrow S = 1$

5. Signal 5 $\rightarrow S = 1 + 1 = 2$

a) Processes that wake up: 3 (all initially blocked processes)

b) Final semaphore value: 2 (2 keys available for future processes)

Part 2: Semaphore Coding

Consider the Producer–Consumer problem using semaphores as implemented in Lab-10 (Lab-plan attached). Rewrite the program in your own coding style, compile and execute it successfully, and explain the working of the code in your own words.

Submission Requirements:

- Your rewritten source code
- A brief description of how the code works
- Screenshots of the program output showing successful execution

Code:

Input:

```

1  #include <stdio.h>
2  #include <pthread.h>
3  #include <semaphore.h>
4  #include <unistd.h>
5
6  #define BUFFER_SIZE 5
7
8  int buffer[BUFFER_SIZE];
9  int in = 0;
10 int out = 0;
11
12 sem_t empty;
13 sem_t full;
14 pthread_mutex_t mutex;
15
16 // Producer
17 void* producer(void* arg)
18 {
19     int id = *(int*)arg;
20
21     for (int i = 0; i < 3; i++)
22     {
23         int item = id * 100 + i;
24
25         sem_wait(&empty); // Wait for space
26         pthread_mutex_lock(&mutex); // Protect buffer
27
28         buffer[in] = item;
29         printf("Producer %d produced item %d at position %d\n", id, item, in);
30         in = (in + 1) % BUFFER_SIZE;
31
32         pthread_mutex_unlock(&mutex);
33         sem_post(&full); // Signal full
34
35         sleep(1);
36     }
37     return NULL;
38 }
39
40 // Consumer
41 void* consumer(void* arg)
42 {
43     int id = *(int*)arg;
44
45     for (int i = 0; i < 3; i++)
46     {
47         sem_wait(&full);
48         pthread_mutex_lock(&mutex);
49
50         int item = buffer[out];
51         printf("Consumer %d consumed item %d from position %d\n", id, item, out);
52         out = (out + 1) % BUFFER_SIZE;
53
54         pthread_mutex_unlock(&mutex);
55         sem_post(&empty);
56
57         sleep(2);
58     }
59     return NULL;
60 }
61
62 int main()
63 {
64     pthread_t prod[2], cons[2];
65     int ids[2] = {1, 2};
66
67     sem_init(&empty, 0, BUFFER_SIZE);
68     sem_init(&full, 0, 0);
69     pthread_mutex_init(&mutex, NULL);
70
71     for (int i = 0; i < 2; i++)
72     {
73         pthread_create(&prod[i], NULL, producer, &ids[i]);
74         pthread_create(&cons[i], NULL, consumer, &ids[i]);
75     }
76
77     for (int i = 0; i < 2; i++)
78     {
79         pthread_join(prod[i], NULL);
80         pthread_join(cons[i], NULL);
81     }
82
83     sem_destroy(&empty);
84     sem_destroy(&full);
85     pthread_mutex_destroy(&mutex);
86
87     return 0;
88 }
89

```

Output:

```
bash - Assignment#2 + - [ ] ...  
kashmirkj@Kashmirpc:~/Operating_System$ ls  
Assignment#1  Evaluation_01  Home_Task_1      Lab05  Lab3    Lab_04  Lab_07  
Assignment#2  Home_Task3      Home_Task_Week10 Lab1    Lab_02  Lab_06  Lab_08  
kashmirkj@Kashmirpc:~/Operating_System$ cd ./Assignment#2/  
kashmirkj@Kashmirpc:~/Operating_System/Assignment#2$ gcc -Lab_Task.c -o Lab_Task -lpthread  
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/13/../../../../x86_64-linux-gnu/Scrt1.o: in function `_start':  
(.text+0x1b): undefined reference to `main'  
collect2: error: ld returned 1 exit status  
kashmirkj@Kashmirpc:~/Operating_System/Assignment#2$ gcc Lab_Task.c -o Lab_Task -lpthread  
kashmirkj@Kashmirpc:~/Operating_System/Assignment#2$ ./Lab_Task  
Producer 1 produced item 100 at position 0  
Consumer 1 consumed item 100 from position 0  
Producer 2 produced item 200 at position 1  
Consumer 2 consumed item 200 from position 1  
Producer 1 produced item 101 at position 2  
Producer 2 produced item 201 at position 3  
Consumer 1 consumed item 101 from position 2  
Consumer 2 consumed item 201 from position 3  
Producer 1 produced item 102 at position 4  
Producer 2 produced item 202 at position 0  
Consumer 2 consumed item 102 from position 4  
Consumer 1 consumed item 202 from position 0  
kashmirkj@Kashmirpc:~/Operating_System/Assignment#2$ =
```

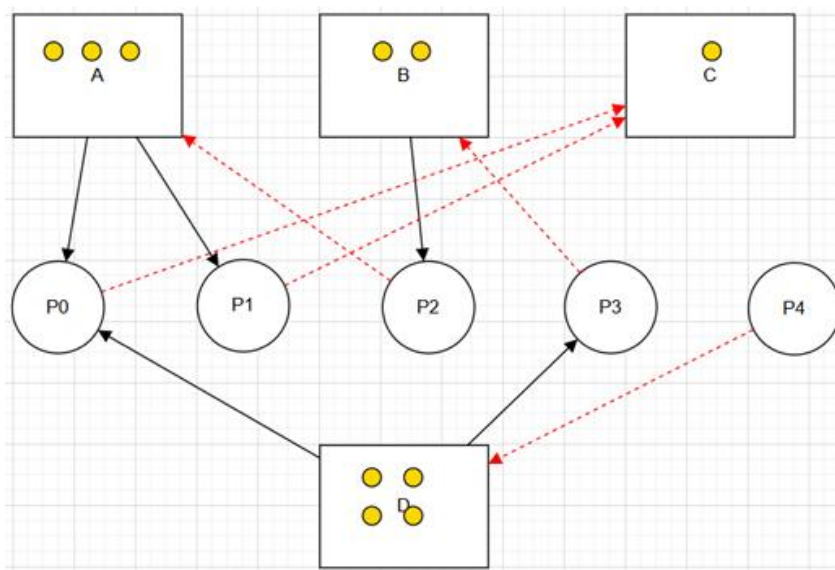
Description:

1. A buffer of size 5 is created to store items.
2. Two indexes are used:
 - in → position where producer puts an item
 - out → position where consumer takes an item
3. Two semaphores are used:
 - empty : keeps track of empty spaces in the buffer
 - full : keeps track of filled spaces in the buffer
4. A mutex is used to ensure that only one thread accesses the buffer at a time.
5. Two producer threads are created.
6. Two consumer threads are created.
7. Each producer produces 3 items and puts them into the buffer.

8. Each consumer consumes 3 items from the buffer.
9. Producers wait if the buffer is full.
10. Consumers wait if the buffer is empty.
11. The buffer works like a circular queue using modulo operation.
12. Producers are faster (sleep(1)), and consumers are slower (sleep(2)).
13. **After all threads finish, semaphores and mutex are destroyed.**

Part 3: RAG (Recourse Allocation Graph)

- Convert the following graph into matrix table ,



Solution:

A. Allocation Matrix (Who HAS what)

Count the solid black arrows coming *out* of a resource box *into* a process circle.

Process	A	B	C	D
P0	1	0	0	1
P1	1	0	0	0
P2	0	1	0	0
P3	0	0	0	1
P4	0	0	0	0
Total Allocated	2	1	0	2

B. Request Matrix (Who WANTS what)

Count the red arrows coming *out* of a process circle *into* a resource box.

Process	A	B	C	D
P0	0	0	1	0
P1	0	1	1	0
P2	1	0	1	0
P3	0	1	0	0
P4	0	0	0	1

C. Available Resources Vector:

A: 3 instances total - 2 allocated = 1 available

B: 2 instances total - 1 allocated = 1 available

C: 1 instance total - 0 allocated = 1 available

D: 4 instances total - 2 allocated = 2 available

Resource	A	B	C	D
Available	1	1	1	2

Part 4: Banker's Algorithm

Part 4: Banker's Algorithm

System Description:

- The system comprises five processes (P0–P3) and four resources (A,B,C,D).
- Total Existing Resources:

Total			
A	B	C	D
6	4	4	2

- Snapshot at the initial time stage:

	Allocation				Max				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	2	0	1	1	3	2	1	1				
P1	1	1	0	0	1	2	0	2				
P2	1	0	1	0	3	2	1	0				
P3	0	1	0	1	2	1	0	1				

Questions:

Questions:

1. Compute the Available Vector:

- Calculate the available resources for each type of resource.

Total resources:

A=6, B=4, C=4, D=2

a) Available matrix

Total allocated:

- A: $2 + 1 + 1 + 0 = 4$
- B: $0 + 1 + 0 + 1 = 2$
- C: $1 + 0 + 1 + 0 = 2$
- D: $1 + 0 + 0 + 1 = 2$

Total - total allocated

- A: $6 - 4 = 2$
- B: $4 - 2 = 2$
- C: $4 - 2 = 2$
- D: $2 - 2 = 0$

Available Vector: [2,2,2,0]

2. Compute the Need Matrix:

- Determine the need matrix by subtracting the allocation matrix from the maximum matrix.

The **Need Matrix** is calculated using the formula: $\text{Need} = \text{Max} - \text{Allocation}$.

Process	Max (A B C D)	Allocation (A B C D)	Need (A B C D)
P0	3 2 1 1	2 0 1 1	1 2 0 0
P1	1 2 0 2	1 1 0 0	0 1 0 2
P2	3 2 1 0	1 0 1 0	2 2 0 0
P3	2 1 0 1	0 1 0 1	2 0 0 0

3. Safety Check:

- Determine if the current allocation state is safe. If so, provide a safe sequence of the processes.
- Show how the Available (working array) changes as each process terminates.

To determine if the state is safe, we find a sequence where each process's **Need** $\leq$ **Available**. Once a process finishes, it releases its **Allocation** back to the **Available** pool.

Step	Process	Need	Available (Work)	Can it run?	New Available (Work + Allocation)
------	---------	------	------------------	-------------	-----------------------------------

1	P0	[1, 2, 0, 0]	[2, 2, 2, 0]	Yes	$[2, 2, 2, 0] + [2, 0, 1, 1] = \{[4, 2, 3, 1]\}$
2	P2	[2, 2, 0, 0]	[4, 2, 3, 1]	Yes	$[4, 2, 3, 1] + [1, 0, 1, 0] = \{[5, 2, 4, 1]\}$
3	P3	[2, 0, 0, 0]	[5, 2, 4, 1]	Yes	$[5, 2, 4, 1] + [0, 1, 0, 1] = \{[5, 3, 4, 2]\}$
4	P1	[0, 1, 0, 2]	[5, 3, 4, 2]	Yes	$[5, 3, 4, 2] + [1, 1, 0, 0] = \{[6, 4, 4, 2]\}$

Yes. **Safe Sequence:** {P0, P2, P3, P1}